

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA0607

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 15

Total Marks:20

Annexure A

Comparative Analysis

Code of Conduct:

I VIMALA K 192424276 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

<i>Criteria</i>	<i>Conceptual Understanding</i>	<i>Accuracy of Answers</i>	<i>Application of Knowledge</i>	<i>Completeness of Response</i>	<i>Clarity & Presentation</i>	<i>Total</i>
<i>Weightage</i>	<i>30%</i>	<i>15%</i>	<i>20%</i>	<i>20%</i>	<i>15%</i>	
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						

Signature of the student:

Total Marks :

Vimala K

Annexure A

Short Answer Test

1. Compare **Linear Regression and Polynomial Regression** for predicting housing prices. Analyze differences in model complexity, ability to capture non-linear relationships, risk of overfitting, and interpretability. Discuss scenarios where each model performs better.

Linear Regression vs Polynomial Regression for Housing Price Prediction

Linear Regression

Linear Regression assumes a **linear relationship** between housing features and price.

Example:

$$\text{Price} = b_0 + b_1 (\text{Size}) + b_2 (\text{Bedrooms}) + b_3 (\text{Location})$$

It produces a straight-line relationship between input and output.

Polynomial Regression

Polynomial Regression extends Linear Regression by adding powers of the features.

Example:

$$\text{Price} = b_0 + b_1 X + b_2 X^2 + b_3 X^3$$

It can model **curved and non-linear relationships**.

Comparison

Factor	Linear Regression	Polynomial Regression
Model complexity	Low	Higher
Relationship	Linear	Non-linear
Flexibility	Low	High
Overfitting risk	Low	Higher, especially with high degree
Interpretability	Very easy	More difficult
Training time	Low	Higher
Captures curves	No	Yes
Suitable for simple data	Excellent	May be unnecessary
Suitable for complex relationships	Limited	Better

Overfitting

Polynomial Regression can become too complex when a high polynomial degree is used. It may fit the training data extremely well but perform poorly on new housing data. This is called **overfitting**.

Linear Regression generally has a lower risk of overfitting because of its simpler structure.

When Linear Regression Performs Better

Linear Regression is preferable when:

- Housing prices have an approximately linear relationship with features.
- The dataset is small.
- Interpretability is important.
- Fast prediction is required.
- The model needs to be easily explained to business users.

When Polynomial Regression Performs Better

Polynomial Regression is preferable when:

- Housing prices have clear curved relationships with features.

- For example, increasing house size may increase price rapidly at first and then have a different effect at larger sizes.
- The dataset contains sufficient observations.
- A moderate polynomial degree can capture the relationship without overfitting.

Conclusion

Linear Regression is simpler, faster, and easier to interpret. **Polynomial Regression** is more flexible and can capture non-linear relationships but has greater complexity and overfitting risk. For a simple housing-price relationship, Linear Regression is usually preferable; when the relationship is clearly curved, Polynomial Regression can provide better predictions.

2. Compare **Recurrent Neural Networks (RNNs)** and **Long Short-Term Memory (LSTM)** networks for time-series prediction tasks such as stock price forecasting. Analyze differences in handling long-term dependencies, vanishing gradient problem, computational complexity, and prediction accuracy.

RNN vs LSTM for Time-Series Prediction

Recurrent Neural Network (RNN)

An RNN processes sequential data by maintaining information from previous time steps.

For example, when forecasting stock prices, an RNN can use previous prices to predict the next price.

However, standard RNNs can have difficulty remembering information from far back in the sequence.

Long Short-Term Memory (LSTM)

LSTM is a specialized type of RNN designed to retain important information for longer periods.

It uses:

- Forget gate

- Input gate
- Output gate
- Cell state

These mechanisms allow LSTM to decide which information should be remembered or discarded.

Comparison

Factor	RNN	LSTM
Architecture	Simple recurrent network	More complex recurrent network
Long-term dependencies	Poorer	Better
Vanishing gradient	More susceptible	Designed to reduce the problem
Short-term patterns	Good	Good
Long-term patterns	Limited	Strong
Computational complexity	Lower	Higher
Training speed	Faster	Slower
Memory capability	Limited	Better
Parameters	Fewer	More
Prediction accuracy	Good for simple sequences	Usually better for complex long sequences

Vanishing Gradient Problem

During RNN training, gradients can become extremely small as they are propagated through many time steps. This is called the **vanishing gradient problem**.

As a result, the RNN may forget information from earlier time steps.

LSTM reduces this problem using its **cell state and gates**, allowing important information to flow through longer sequences.

Computational Complexity

A basic RNN has a simpler structure and fewer parameters, so it generally requires less computational power.

LSTM contains multiple gates and additional parameters, making it computationally more expensive and slower to train.

Prediction Accuracy

For short and relatively simple time-series patterns, an RNN can provide good results.

For complex sequences with long-term dependencies, LSTM often performs better because it can retain important historical information.

However, **LSTM is not automatically more accurate in every stock-price forecasting problem**. Stock prices are highly noisy and influenced by many external factors, so model performance depends heavily on the data, features, preprocessing, and forecasting horizon.

When RNN Performs Better

RNN is suitable when:

- The sequence is relatively short.
- Dependencies are mainly recent.
- The problem is simple.
- Faster training is preferred.
- Computational resources are limited.

When LSTM Performs Better

LSTM is suitable when:

- Long-term dependencies are important.
- The sequence is long.
- Historical information from distant time steps affects predictions.
- The data contains complex temporal patterns.
- Higher computational cost is acceptable.

Conclusion

RNN is simpler and computationally cheaper but can struggle with long-term dependencies and vanishing gradients. **LSTM** is more complex and computationally expensive but handles long-term dependencies more effectively. For complex time-series forecasting tasks, LSTM is generally a stronger choice, while RNN can be sufficient for simpler sequential problems.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand